

Arquitectura e implementación de un agente RAG para películas

Proyecto final del curso de Inteligencia Artificial

Grupo 1

1. Resumen

Este proyecto presenta la implementación de un sistema RAG (*Retrieval-Augmented Generation*) aplicado a un catálogo de películas, desarrollado como proyecto final del curso de Inteligencia Artificial. El objetivo principal fue construir un prototipo funcional capaz de responder consultas en lenguaje natural utilizando recuperación semántica y generación de respuestas mediante un modelo LLM ejecutado localmente.

La arquitectura propuesta integra distintas etapas del flujo RAG. Primero, se obtiene y almacena información de películas en una base de datos SQLite mediante procesos de *web scraping*.

Posteriormente, se generan embeddings semánticos utilizando el modelo sentence-transformers/all-MiniLM-L6-v2, tanto para el corpus de películas como para las consultas realizadas por el usuario. Estos vectores permiten calcular similitud coseno y recuperar los documentos más relevantes utilizando un enfoque Top-K.

La información recuperada es utilizada para construir dinámicamente un prompt contextual que se envía al modelo llama3 mediante Ollama. Este enfoque permite que el sistema genere respuestas más relevantes y reduzca significativamente el riesgo de alucinaciones, ya que el modelo responde únicamente utilizando el contexto recuperado desde la base de datos.

Finalmente, el sistema fue evaluado mediante métricas estándar para arquitecturas RAG, incluyendo Accuracy@1, Recall@3, MRR, Faithfulness y Answer Relevancy, obteniendo resultados consistentes para un prototipo académico y demostrando la viabilidad de combinar recuperación semántica y generación aumentada en un entorno local.

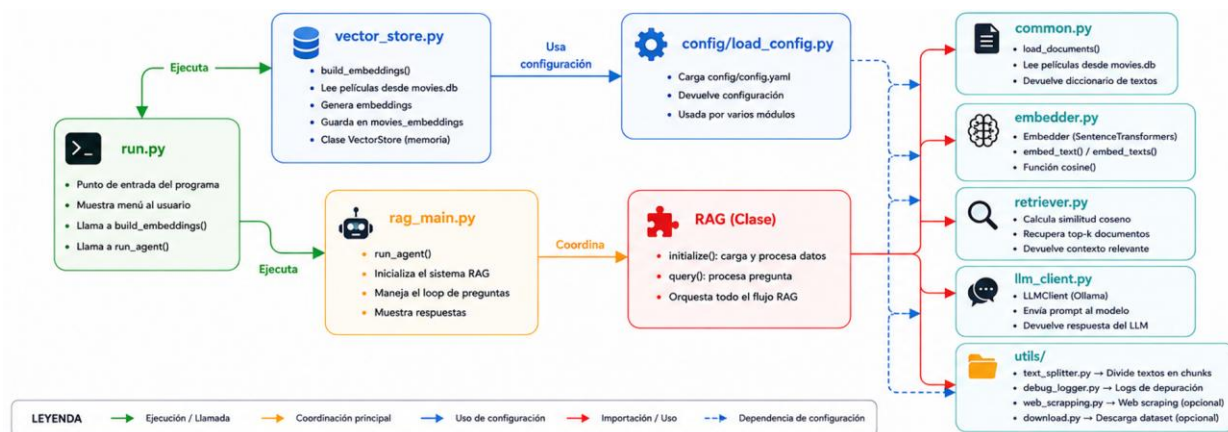
2. Diseño del sistema

La estructura del proyecto está organizada en carpetas y scripts que separan la configuración, la obtención de datos, la vectorización, la recuperación y la generación de respuestas.

- config/: contiene config.yaml y load_config.py para centralizar parámetros del sistema.
- data/: almacena movies.db, queries.db y el archivo con consultas de evaluación.
- utils/: reúne utilidades como web_scrapping.py y common.py.
- install.bat y start.bat: automatizan la instalación y el arranque del proyecto.
- run.py: es el punto de entrada principal del sistema.

El servidor LLM se ejecuta localmente con Ollama. Para este proyecto se usó llama3 por ser un modelo ligero y adecuado para el hardware disponible, aunque la arquitectura permite cambiarlo según las capacidades de la máquina.

Estructura de Conexión:



3. Embeddings

3.1 Embedder

El archivo embedder.py transforma texto natural en vectores numéricos. Se utilizó el modelo sentence-transformers/all-MiniLM-L6-v2, elegido por su buen equilibrio entre calidad semántica y costo computacional. Cada texto se convierte en un vector de 384 dimensiones y se normaliza para facilitar el cálculo posterior de similitud.

- La misma instancia del modelo se usa para indexar las películas y para vectorizar la consulta del usuario.
- El módulo expone funciones para generar embeddings de un texto o de varios textos en batch.

3.2 Vector Store

El archivo vector_store.py lee los registros de la tabla movies, combina movie_name y description en un texto semántico, genera el embedding y lo guarda en movies_embeddings. En esta implementación, SQLite cumple el papel de almacén vectorial local, lo cual es suficiente para un prototipo académico y un conjunto pequeño de datos.

- La información se serializa como BLOB para persistir el vector en la base de datos.
- Cada embedding queda asociado al movie_id para mantener trazabilidad con la película original.
- La solución puede migrarse en el futuro a FAISS o ChromaDB sin cambiar la lógica general del sistema.

4. RAG

4.1 Recuperación

Cuando el usuario realiza una consulta, el sistema la convierte en embedding con el mismo modelo utilizado en el corpus. Después se calcula la similitud de cosenos contra los 126 vectores almacenados en SQLite y se seleccionan los Top-K resultados, con $k = 3$.

4.2 Prompt y generación

El sistema implementa una estrategia de Prompt Engineering orientada a reducir alucinaciones y asegurar que el modelo responda únicamente utilizando la información recuperada desde la base de datos.

El prompt actúa como una capa de control entre la etapa de recuperación semántica y la generación de lenguaje natural, proporcionando al modelo instrucciones explícitas, contexto recuperado y la consulta original del usuario.

El flujo implementado es el siguiente:

Retrieved Context

+

User Query

+

System Instructions

↓

Prompt Engineering Layer

↓

LLM Response

La construcción del prompt se realiza dinámicamente dentro del pipeline RAG. Primero, el sistema recupera los Top-K documentos más relevantes utilizando similitud coseno sobre los embeddings almacenados. Posteriormente, estos documentos se concatenan como contexto y se envían junto con la pregunta del usuario al modelo llama3 ejecutado mediante Ollama.

El prompt implementado en el sistema es el siguiente:

```
prompt = f"""
You are a movie expert assistant.

There are {len(context_docs)} relevant documents in the movie database that may
contain the answer to the user's question.

Answer the user's question based on the retrieved movie database context.

Respond using only the context retrieved from the movie database.

Do not use any external knowledge or assumptions.

Avoid making up information and do not give questions back to the user.

If the answer is not explicitly found in the context, respond with:
"I cannot answer based on the available database."

### Retrieved Context:
{context_text}

### User Question:
```

```
{question}

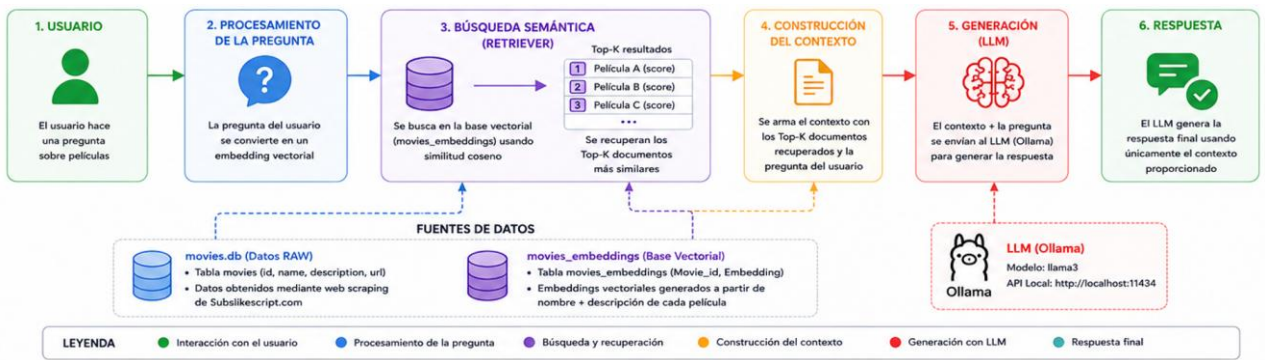
### Answer:
"""
```

Este diseño obliga al modelo a generar respuestas condicionadas exclusivamente al contexto recuperado, minimizando el uso de conocimiento externo y reduciendo significativamente el riesgo de respuestas inventadas.

Además, el prompt incorpora mecanismos de control importantes: restricción explícita de conocimiento externo, delimitación clara del contexto, estructura guiada de entrada y salida, y manejo de casos donde la información no existe en la base de datos.

Gracias a esta estrategia, el sistema logra respuestas más coherentes, interpretables y alineadas con la información disponible en el catálogo de películas.

Arquitectura RAG:



5. Estructura de almacenamiento

La base de datos principal contiene dos tablas relevantes: movies, donde se almacena el catálogo, y movies_embeddings, donde se guardan los vectores generados. Esta organización mantiene el sistema simple, portátil y fácil de reproducir.

6. Evaluación

La evaluación se realizó con 20 preguntas de prueba y cubrió tanto la calidad de recuperación como la calidad de la respuesta generada. Se usaron métricas estándar para sistemas RAG.

Métrica	Resultado
Accuracy@1	0.65
Recall@3	0.75
MRR	0.70
Faithfulness	0.88
Answer Relevancy	0.89
RAG Global Score	0.77

Los resultados muestran que el sistema recupera la película correcta en el top 3 la mayor parte del tiempo y que las respuestas del LLM se mantienen fieles y relevantes al contexto proporcionado.

7. Conclusiones y mejoras futuras

- El sistema es un prototipo sólido para un proyecto académico y demuestra el funcionamiento completo de un flujo RAG.
- La combinación de embeddings semánticos, recuperación por similitud y generación con contexto reduce el riesgo de respuestas no fundamentadas.
- Como mejoras futuras, se recomienda enriquecer la base de datos, dividir descripciones en chunks y explorar una base vectorial especializada.
- También sería posible extender el sistema hacia búsquedas multimodales con imágenes, audio o video.

8. Participación

Jairo Cabrera participó en el diseño general del sistema y en la configuración inicial del proyecto. Se encargó de la integración del servidor LLM mediante Ollama, la preparación del entorno de ejecución utilizando install.bat y start.bat, así como de la estructura base de datos y del módulo de web scraping utilizado para obtener la información de películas.

Nicolás Carrión desarrolló la capa de embeddings y recuperación semántica. Implementó los módulos embedder.py y vector_store.py, integrando el modelo sentence-transformers/all-MiniLM-L6-v2 para la generación de embeddings y el almacenamiento vectorial en SQLite.

Evelyn Bermeo estuvo a cargo de la lógica principal del flujo RAG y del diseño del prompt. Participó en la vectorización de consultas del usuario, recuperación contextual, construcción del prompt dinámico y comunicación con el modelo llama3 mediante requests hacia Ollama, asegurando respuestas contextualizadas y controladas. También trabajó en la implementación de similitud coseno y recuperación Top-K.

Javier Cuzco fue responsable de la evaluación experimental del sistema. Diseñó el conjunto de pruebas y trabajó en la implementación y análisis de métricas como Accuracy@1, Recall@3, MRR, Faithfulness y Answer Relevancy, interpretando los resultados obtenidos para validar el desempeño general del sistema RAG.

El desarrollo del proyecto se realizó de manera colaborativa, con participación conjunta en el análisis, la implementación, la evaluación y la revisión final de la propuesta.